Dhaka Urban Management System Lab Examination

Project 2 Questions and Model Solutions

Complete all five phases. Write Phases 1–4 on paper and implement Phase 5 in Java. This document provides an examination paper without marks, followed by a model solution. Alternative designs are acceptable when their analysis, diagrams, and implementation are consistent.

Part A Examination Questions

Project scenario

Design a Dhaka Urban Management System that integrates urban services, citizen communication, IoT monitoring, and decision support. The broader platform aims to manage traffic, improve waste collection, handle complaints, support emergency response, monitor environmental conditions, and generate urban reports.

For the examination, implement a complaint-to-service workflow: a citizen submits a complaint, the system creates the appropriate urban service, selects a response route using external traffic data, updates the complaint status, and notifies subscribers.

Instructions

- Complete Phases 1–4 on paper and Phase 5 as a Java console application.
- Simulate complaints, routes, traffic information, and notifications. A live database, internet service, physical sensors, and a graphical interface are not required.
- Use all five specified patterns. Clearly distinguish the broader platform from the implemented subsystem.
- Use consistent class names and state reasonable assumptions.

Phase 1 Requirement Analysis

Question 1 Analyze the proposed system

- Describe the system background and identify three problems the proposed system addresses.
- State one primary objective and four specific objectives.
- Identify at least four actors and their responsibilities. Define the boundary between the platform and external devices or services.
- Specify four functional requirements and two non-functional requirements. At least two functional requirements must concern complaint handling and service dispatch.

Phase 2 Object Identification

Question 2 Identify entity control and boundary objects

- Identify at least five entity classes, giving two or more attributes for each.
- Identify three control classes and explain their responsibilities.
- Identify three boundary classes and explain their external interactions.
- Present the classifications in tables and explain any difference between an external actor and its internal software representation.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

Prepare six CRC cards for the complaint-handling and service-dispatch workflow. Include class name, at least two responsibilities, and collaborating classes. Across the six cards, include at least one entity, one control, and one boundary class.

Phase 4 UML Modeling

Question 4 Model the system structure and behavior

a. Draw a UML class diagram with at least eight classes or interfaces. Include important attributes, methods, visibility indicators, relevant associations and multiplicities, and interface realizations. Show the participants in the selected patterns. Use aggregation or composition only when ownership and lifecycle semantics justify it.

b. Draw a sequence diagram for dispatching a response to an OPEN complaint. Show complaint retrieval, service creation, external traffic retrieval through an adapter, route selection, the status change to IN_PROGRESS, and observer notification. Use an alt fragment for a route available versus no route available. When no route exists, the complaint must remain OPEN.

Phase 5 Java Implementation

Question 5 Implement and demonstrate five design patterns

Pattern	Required implementation
Singleton	UrbanDatabase provides controlled access to one shared in-memory complaint repository. Demonstrate instance identity.
Factory	UrbanServiceFactory creates WasteService, TrafficService, or EmergencyService through UrbanService. Reject unknown types. A Simple Factory is acceptable; name the variant.
Observer	Publish complaint status changes to CitizenNotifier and GovernmentDashboard. Demonstrate subscription, removal, and citizen-specific notification filtering.
Strategy	RoutePlanner switches between ShortestRouteStrategy and FastestRouteStrategy at runtime.
Adapter	TrafficAPIAdapter translates a simulated external traffic API into the TrafficServiceAPI interface, converting metres to kilometres and seconds to minutes.

Examination rules

Use status transitions OPEN → IN_PROGRESS → RESOLVED. Submission creates OPEN; dispatch changes OPEN to IN_PROGRESS only after a route is obtained; resolution is allowed only from IN_PROGRESS. Reject duplicate complaint IDs, missing IDs, unsupported service types, and invalid transitions.

Choose among three candidate routes to the same destination. Route A: 5 km, 30 minutes; Route B: 8 km, 20 minutes; Route C: 6.5 km, 25 minutes. Shortest chooses minimum distance; fastest chooses minimum estimated time. Break ties by route ID in ascending lexical order. Reject an empty candidate list.

Required Demonstrations and Submission

Test	Input or action	Expected behavior
1	Request repository twice	Same instance
2	Create WASTE, TRAFFIC, EMERGENCY	Correct concrete service types
3	Request UNKNOWN service	Reject request
4	Submit valid complaint	Stored with OPEN status
5	Repeat ID or use blank ID	Reject without a new complaint
6	Convert route A provider values	5000 m → 5 km; 1800 s → 30 min
7	Switch shortest to fastest	A first, then B
8	Dispatch valid OPEN complaint	Selected route; IN_PROGRESS; observers notified
9	Resolve an OPEN complaint	Reject; remains OPEN
10	Unsubscribe dashboard then resolve	RESOLVED; citizen notified, dashboard not notified
11	Dispatch RESOLVED or missing ID	Reject; no valid transition
12	No candidate routes during dispatch	Reject; complaint remains OPEN

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and test results showing expected and actual behavior. The candidate-route values above are fictional examination data, not observations of Dhaka traffic.